

Seguridad de subida de ficheros

Objetivos de aprendizaje

Separar aceptación, almacenamiento, recuperación, interpretación y ejecución; identificar qué metadato o contenido decide cada capa y confirmar una capacidad cada vez.

Prerrequisitos

Multipart, MIME, rutas, servidores estáticos, handlers, autorización y parsers de formatos.

Fundamentos técnicos

Una subida correcta solo demuestra que la aplicación aceptó bytes. El impacto depende de pasos posteriores:

```
subida -> validación -> nombre/ubicación -> almacenamiento -> recuperación
      -> cabeceras/handler -> interpretación -> posible ejecución
```

El Content-Type de la parte, el nombre y la extensión son datos del cliente; la defensa puede usarlos como una señal, pero debe validarlos junto al contenido y aislar el almacenamiento. Ningún campo aislado decide universalmente si un fichero se ejecuta.

Modelo mental

Para cada prueba registra: nombre declarado, MIME declarado, firma/contenido, nombre final, ubicación, URL, cabeceras de descarga y componente que procesa los bytes.

Superficie de ataque

Avatares, adjuntos, importaciones, temas, documentos, archivos comprimidos y endpoints PUT. Cada flujo puede almacenar en disco local, objeto remoto o cola de procesamiento.

Cómo identificarla

- La respuesta entrega un identificador o nombre final.
- El fichero puede recuperarse con identidad y cabeceras conocidas.
- El contenido se sirve estático, se descarga como adjunto o pasa por un parser.
- Solo una respuesta del handler o un marcador interpretado confirma ejecución server-side.

Preguntas que debo hacerme

1. ¿Qué controles se aplican y en qué orden?
2. ¿Se renombra y dónde se almacena?
3. ¿Quién puede recuperar el objeto?
4. ¿Qué Content-Type y Content-Disposition devuelve el servidor?
5. ¿Qué handler, si alguno, interpreta la extensión o bytes?

Prueba mínima

Subir un fichero de texto con token único, recuperarlo mediante el flujo normal y comparar hash, nombre y cabeceras. Esa prueba no intenta ejecución.

Construcción progresiva del payload

1. Texto marcador permitido.
2. Mismo contenido con un metadato cambiado.
3. Mismo nombre con contenido inválido para el tipo.

4. Localizar nombre final y política de acceso.
5. Comparar descarga forzada y renderizado.
6. Solo en un laboratorio preparado, usar un marcador interpretado inocuo para comprobar handler.

Anatomía de los payloads

- Contexto de entrada: parte multipart file.
- Sintaxis original: nombre probe.txt, tipo text/plain, bytes UPLOAD-7F31.
- Entrada controlada: metadatos y bytes de la parte.
- Transformaciones conocidas: parser multipart, renombrado UUID.
- Parser final: almacenamiento de objetos; sin intérprete.
- Sink: escritura y posterior descarga.
- Primitiva: persistir y recuperar bytes idénticos.
- Payload mínimo: fichero marcador de texto.
- Significado de cada componente: token único permite atribución y hash.
- Resultado esperado: 201, ID y descarga idéntica.
- Control negativo: ID inexistente y acceso de otra cuenta.
- Restricción observada: extensiones no permitidas se renombran .bin.
- Por qué falla la variante básica: cambiar solo Content-Type no altera nombre final ni handler.
- Hipótesis de adaptación: estudiar la capa que realmente decide interpretación.
- Payload adaptado: no procede si el objeto se sirve con descarga forzada y sin handler.
- Por qué debería funcionar: solo habría ejecución si una ruta posterior interpretase los bytes.
- Evidencia: cabeceras application/octet-stream, attachment y ausencia de handler.
- Cuándo no funcionaría: procesamiento posterior, servidor mal configurado o ubicación ejecutable.

Variaciones según el contexto

Un SVG puede ser seguro como descarga y peligroso si se renderiza same-origin. Un ZIP puede ser seguro almacenado y peligroso al extraerse sin validar rutas. Una imagen puede activar vulnerabilidades del parser aun sin ejecución por extensión.

Filtros y bypasses

Cambiar extensión, MIME y bytes a la vez impide saber qué control falló. Aísla cada variable. Una allowlist de extensión es una capa; no sustituye renombrado, aislamiento, límites, autorización y desactivación de handlers.

Evidencias de confirmación

Aceptación, persistencia, recuperación e interpretación requieren evidencias separadas. Un 201 no confirma ubicación pública; una URL pública no confirma ejecución; un 200 puede ser descarga estática.

Escalado de impacto

Evaluar acceso entre usuarios, overwrite, path traversal en nombre, serving same-origin, extracción y parsers. Solo probar ejecución server-side en entorno preparado y con marcador no destructivo.

Errores frecuentes

- Titular la técnica “subida a shell”.
- Confiar en Content-Type enviado por el cliente.

- Probar doble extensión sin saber qué componente selecciona handler.
- Buscar rutas por fuerza bruta cuando existe un ID.
- Confundir contenido PHP servido como texto con ejecución.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
201, pero URL original da 404	renombrado o almacenamiento externo	usar ID devuelto y endpoint de descarga
bytes cambian	transcodificación/CDR	comparar hash, dimensiones y metadatos
extensión ejecutable se descarga	sin handler o attachment	marcador interpretado seguro y cabeceras
MIME falso se rechaza	inspección de contenido	cambiar solo bytes con extensión permitida
otra cuenta recibe 403	autorización por objeto	matriz identidad-ID-acción
fichero desaparece	cuarentena, TTL o antivirus	observar estado asíncrono y tiempos

Mitigaciones

Allowlist necesaria, validación de firma/contenido, nombre generado, límites, autorización, almacenamiento fuera del webroot o en otro host, serving mediante handler seguro, nosniff, descarga cuando proceda y análisis/CDR según riesgo.

Relación con pentesting y certificaciones

Se evalúa el pipeline demostrado, no si se obtuvo una shell.

Caso guiado

Caso A — Básico: adjunto recuperable

Un .txt con token recibe ID 8c2; GET /files/8c2 devuelve bytes idénticos con Content-Disposition: attachment.

Observación: aceptación, almacenamiento y recuperación confirmados. Qué sé: no hay interpretación. Hipótesis: objeto privado o público. Experimento: sin sesión y con otra cuenta. Resultado: ambos 403. Conclusión: flujo básico autorizado; no existe vulnerabilidad demostrada. Siguiendo paso: revisar controles de tipo y tamaño por separado.

Caso de adaptación

Caso B — El nombre se conserva, pero no se ejecuta

El laboratorio acepta probe.phtml y permite descargarlo. El cuerpo PHP aparece literal.

Observación: extensión conservada y contenido recuperable. Hipótesis: handler PHP o serving estático. Experimento: marcador que, si se interpretara, produciría texto distinto; revisar cabeceras y configuración de ruta. Resultado: bytes literales, application/octet-stream, host de objetos separado. Conclusión: no hay ejecución; adaptar extensión no cambia el componente. Siguiendo paso: documentar almacenamiento potencialmente riesgoso solo según consumidores posteriores.

Caso C — Transferencia: paquete de traducción

Una función acepta ZIP y lo extrae. Un archivo con ruta ../marker.txt aparece fuera del directorio de traducciones del laboratorio.

Resolución: upload -> almacenamiento temporal -> parser ZIP -> normalización de rutas -> escritura. La primitiva transferida no es ejecución: es path traversal durante extracción. Se confirma con ruta interna normal y marcador fuera de base.

Caso D — Falso positivo: respuesta “uploaded”

El frontend muestra “uploaded”, pero la respuesta HTTP es 422 y no existe ID; el mensaje se genera antes de esperar al servidor.

Observación: éxito visual. Hipótesis: persistencia o UI optimista. Experimento: respuesta cruda, recarga y listado. Resultado: no hay objeto. Conclusión: ni siquiera se confirmó subida.

Ejercicios

1. Diseña una matriz que cambie solo extensión, MIME o bytes.
2. Propón evidencia independiente para cada flecha del pipeline.
3. Distingue serving estático, descarga y handler con cabeceras y marcador.
4. Analiza un ZIP sin asumir que el riesgo principal es ejecución.

Resumen

Subir no implica almacenar, recuperar, interpretar ni ejecutar. El análisis seguro confirma cada transición y atribuye el comportamiento al componente correcto.

Chuleta operativa

1. Nombre, MIME, bytes y tamaño.
2. ID/nombre final y ubicación.
3. Hash recuperado.
4. Autorización.
5. Cabeceras de serving.
6. Parser/handler.
7. Una variable por prueba.

Referencias

- [PortSwigger: file upload](#)
- [OWASP File Upload Cheat Sheet](#)
- [Concepto interno](#) (file-upload)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Práctica: [cuaderno](#).